FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Luis Gustavo Bonfim Ciaramicoli

Produto: **PostCraft AI**
https://github.com/luisciaramicoli/postcraft-ai

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 12/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 6
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 7
4 Quadro de entregas 7
5 Atividades realizadas 8
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 8
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 10
5.3 Atividade 9 — Fork e plano de negócio 12
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 13
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 16
6 Considerações finais 22
7 Declaração de autoria 22

1 Introdução

Introdução à Computação em Nuvem (ISW055), na Fatec Pompeia, propõe que o aluno aprenda nuvem construindo e operando um sistema de verdade: containers com Docker, serviços separados que conversam por uma rede interna, autenticação e autorização no servidor, logs de auditoria, armazenamento de objetos, cobrança com Stripe e publicação em um domínio. Em vez de exercícios isolados, cada atividade acrescenta uma peça ao mesmo projeto, e este relatório consolida o segundo bimestre.

O projeto começou como o CineCloud, um catálogo de filmes com cadastro, login, favoritos e comentários. Nas atividades 7 e 8 ele ganhou um plano pago com Stripe e uma documentação reversa. Na atividade 9 o repositório foi copiado com o histórico para um repositório novo, o do PostCraft AI, e nas atividades 10 e 11 o domínio de filmes foi trocado por um negócio próprio: um gerador de carrosséis e legendas para Instagram, voltado a profissionais autônomos (nutricionistas, advogados, personal trainers, psicólogos) que precisam publicar com regularidade, mas não têm tempo para planejar roteiros, escrever legendas e manter uma identidade visual.

O produto oferece um plano gratuito, limitado a três posts gerados por mês, e uma assinatura chamada Creator Pro, de R$ 49,90 por mês, sem limite de gerações. Para essa promessa ser verdadeira, o servidor precisa saber quem é o usuário, qual é o seu papel, a quem pertence cada registro e se a assinatura está ativa. Este relatório mostra a arquitetura que existe hoje no repositório e na stack, as cinco entregas das atividades 7 a 11 com as respectivas evidências e, com a mesma clareza, aquilo que ainda não está pronto.

O documento está organizado como pede o enunciado: metodologia, arquitetura do SaaS em oito partes, quadro de entregas gerado a partir das fichas, uma ficha por atividade, considerações finais e declaração de autoria.

2 Metodologia

As atividades 7 e 8 foram feitas no repositório do catálogo de filmes, o mesmo do primeiro bimestre. Na atividade 9, o histórico desse repositório foi preservado no repositório novo `luisciaramicoli/postcraft-ai`, onde o SaaS foi desenvolvido nas atividades 10 e 11. Por isso os commits das atividades 7 e 8 aparecem com o mesmo hash nos dois repositórios. O desenvolvimento usou Docker Compose localmente e a stack do Portainer para publicação, e o código das imagens é construído pelo GitHub Actions e publicado no GHCR.

Cada data e hora de entrega vem do histórico Git (`git log`) e foi conferida na página do commit no GitHub. Da atividade 9 em diante, usei também a data de abertura da issue de entrega que menciona o professor, convertida de UTC para o horário de Brasília (UTC−03:00). Os prints mostram páginas reais do GitHub, o sistema em execução e, nas fichas, um registro textual com o hash e o horário absolutos, porque o GitHub mostra datas relativas. As capturas feitas por mim em 12/11/2026 trazem uma faixa com data e hora de captura.

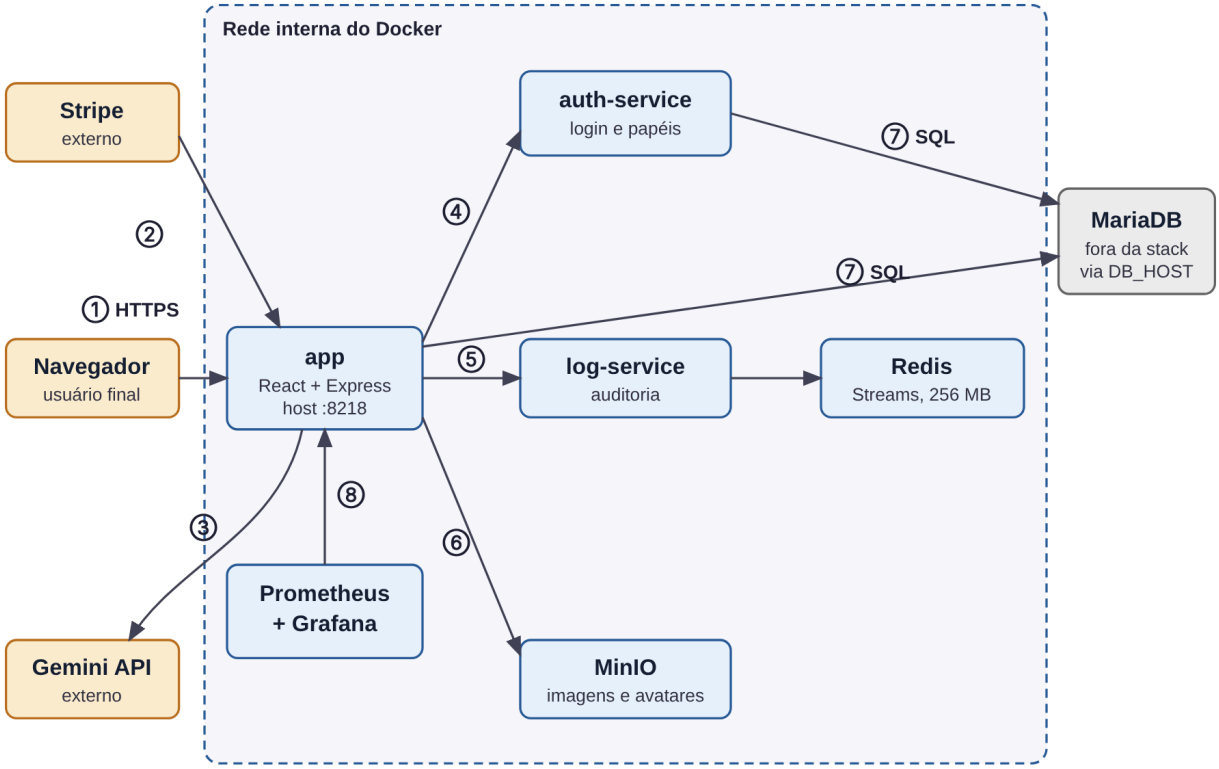
Há duas ressalvas de método. Primeiro, a URL pública do produto foi consultada em 12/11/2026, às 21:56, e o nome da stack no Portainer e a identificação da VM não puderam ser confirmados a partir dos arquivos; por isso não são afirmados aqui. Segundo, os testes de 403 e de eventos no log (atividade 10) e o ciclo de assinatura (atividade 11) foram feitos numa instância local, construída com o código mais recente, em contas de teste e com o Stripe em modo de teste; o webhook, porém, foi entregue pelo próprio Stripe. Data de código ou de issue não prova aprovação do professor nem o funcionamento de tudo o que a interface anuncia.

A redação e a montagem deste documento tiveram assistência de IA, que também executou parte dos testes descritos. Nenhum registro de pagamento, publicação ou entrega foi inventado: o que não foi comprovado está marcado como pendência.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O diagrama mostra o sistema como ele está no repositório e no Compose. O navegador chega ao app pelo domínio público; os demais serviços do Compose só se enxergam pela rede interna do Docker, por nome de serviço. O Stripe é o único serviço externo que chama o app, por meio do webhook.



Laranja: externo ou público. Azul: Compose, só na rede interna. Cinza: banco fora do Compose. ① navegador → app pelo domínio · ② Stripe → app (webhook) · ③ app → Gemini · ④ app → auth-service (papel) · ⑤ app → log-service → Redis · ⑥ app → MinIO · ⑦ app e auth → MariaDB · ⑧ Prometheus coleta métricas do app; Grafana lê o Prometheus.

Figura 1: Diagrama de componentes e microsserviços do PostCraft AI

Componente	Responsabilidade	Tecnologia / imagem	Público?
app	telas (React), API do negócio (posts, marcas, perfil), landing page, checkout e webhook, geração de conteúdo via Gemini	Node 20, Express, Vite; `postcraft-app`	sim (domínio, porta 8218)
auth-service	cadastro, login, sessão e papéis (RBAC)	Node, Express; `postcraft-auth`	não (rede interna)
log-service	eventos de negócio e auditoria (CloudEvents)	Node, Express; `postcraft-log-service`	não (rede interna)

Componente	Responsabilidade	Tecnologia / imagem	Público?
Redis	armazenamento dos logs (Redis Streams, 256 MB)	`redis:7-alpine`	não (rede interna)
MariaDB	usuários, marcas, posts, templates e dados de assinatura	MariaDB fora do Compose, acessado por `DB_HOST`	não (só a stack acessa)
MinIO	fotos de perfil, logos e imagens de posts	MinIO (imagem do projeto no GHCR)	porta não publicada; bucket de leitura pública
Stripe (externo)	checkout, assinaturas, webhooks	serviço SaaS	webhook → app
Gemini (externo)	texto e imagem dos posts	API do Google	app → Gemini
Prometheus e Grafana	métricas e painéis	`prom/prometheus`, `grafana/grafana`	Prometheus só em 127.0.0.1

3.2 Onde roda

O produto é servido em https://luis-ciaramicoli-isw055.lapps.studio/, o mesmo domínio usado desde o início do semestre, com o app exposto na porta 8218 do host. O código vem do repositório `luisciaramicoli/postcraft-ai`: o Compose de produção (`docker-compose.prod.yml`) referencia as imagens `postcraft-app`, `postcraft-auth` e `postcraft-log-service` no GHCR, construídas pelo GitHub Actions, além de `redis:7-alpine`, MinIO, Prometheus, Grafana e Watchtower. Os containers são o app, o auth-service, o log-service, o Redis, o MinIO, o Prometheus, o Grafana e o Watchtower.

Os arquivos Compose não definem limite de memória por container; o único limite é o `maxmemory` de 256 MB do Redis, que vale apenas para os dados dele. A afirmação de 1 GB feita na issue 8 não aparece nos arquivos e não é tratada como configuração comprovada. O nome da stack no Portainer e a identificação da VM também não constam nos arquivos do repositório e não foram verificados por mim.

Variáveis de ambiente (somente os nomes): `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `MINIO_ENDPOINT`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_BUCKET`, `STRIPE_SECRET_KEY`, `STRIPE_PRICE_ID`, `STRIPE_WEBHOOK_SECRET`, `GEMINI_API_KEY`, `GEMINI_TEXT_MODEL`, `GEMINI_IMAGE_MODEL`, `SMTP_USER`, `SMTP_PASS`, `PUBLIC_URL`, `COOKIE_SECURE` e `CORS_ORIGINS`. O Compose de produção traz valores padrão inseguros para alguns segredos, que precisam ser substituídos na stack; os valores não são reproduzidos aqui.

3.3 Fluxo de uma requisição

Ação escolhida: o criador gera um post com IA. (1) O navegador envia `POST /api/posts/generate` ao domínio, que entrega ao app; o cookie HttpOnly com o JWT acompanha a requisição. (2) O middleware valida o token e consulta o auth-service para confirmar o papel: `criador` e `admin` podem criar, `revisor`

recebe 403. (3) O controlador conta no MariaDB os posts gerados pelo usuário no mês corrente; se o usuário não é premium e já tem três, responde 403 com o código `limite_gratuito_atingido`, antes de chamar o Gemini. (4) Caso contrário, o serviço de geração chama o Gemini (texto e imagem) e o controlador grava o post com `usuario_id` da sessão. (5) O log-service recebe o evento de negócio e o grava no Redis Streams. (6) A API responde 201 com o post. Nas listagens, usuários comuns recebem apenas registros filtrados por `usuario_id`; o administrador tem um caminho específico para consulta ampliada.

3.4 Fluxo de assinatura

O fluxo parte do botão "Fazer Upgrade para Creator Pro" no perfil. O app cria uma Checkout Session em `mode=subscription` com o preço configurado, o `client_reference_id` do usuário e a metadata `userId`; o usuário paga na página hospedada do Stripe, e o app nunca recebe dados de cartão. A confirmação depende do webhook, não do retorno do navegador: o app valida a assinatura do evento com `constructEvent`, usando o corpo bruto e `STRIPE_WEBHOOK_SECRET`, e rejeita com 400 quando o cabeçalho falta ou a assinatura não confere. O evento `checkout.session.completed` grava `premium=1`, `stripe_customer_id` e `stripe_subscription_id` e registra um evento de auditoria; `customer.subscription.deleted` zera `premium` pelo `stripe_subscription_id`. A cota é aplicada na API de geração, que ignora o limite para usuários premium.



Figura 2: Fluxo de assinatura: checkout, webhook, plano no banco e limite

Três lacunas estão declaradas. Não existe rota do portal do cliente, então o cancelamento foi testado pela API do Stripe e não pelo portal. Não há armazenamento dos IDs de evento: se o mesmo evento chegar duas vezes, o `UPDATE` mantém o mesmo valor, mas isso não é idempotência completa, pois efeitos secundários, como o evento de auditoria, se repetem. E os eventos `customer.subscription.updated` e `invoice.payment_failed` não são tratados. A validação ponta a ponta do ciclo está na ficha da atividade 11.

3.5 Modelo de dados do domínio

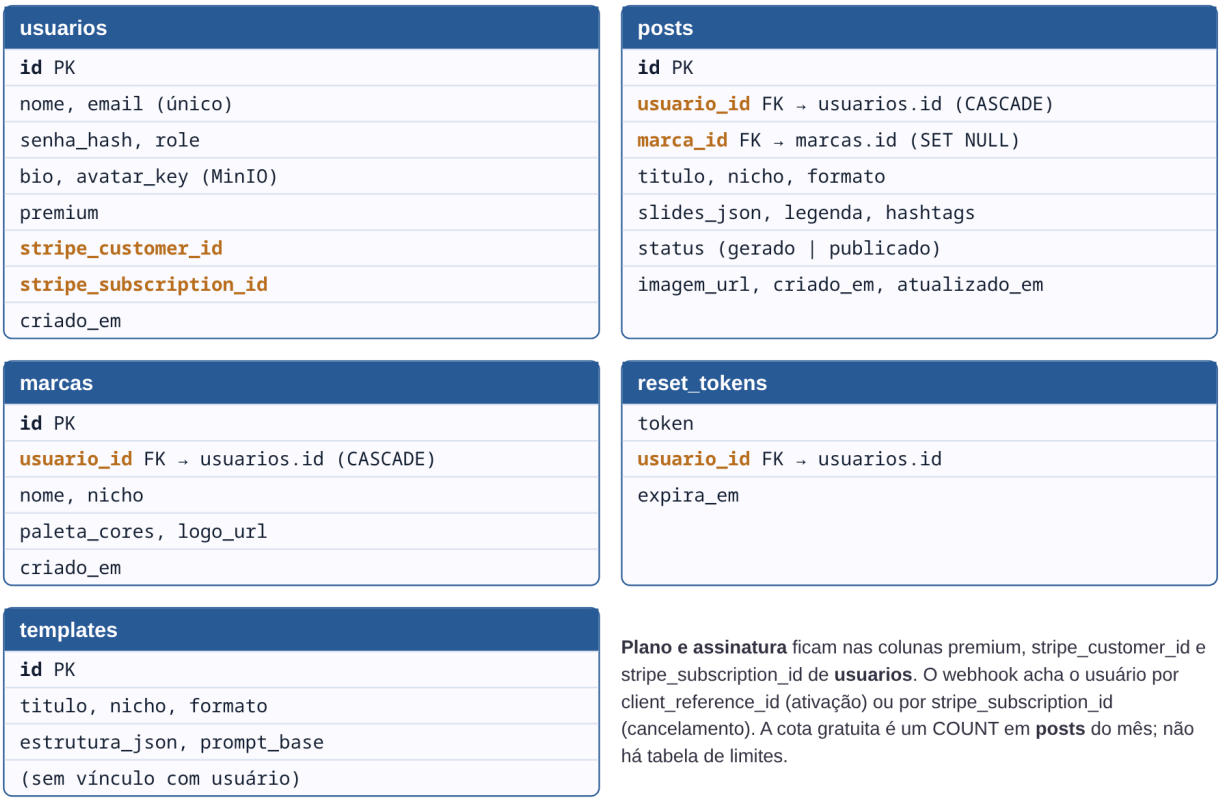


Figura 3: Modelo de dados do domínio

O vínculo entre usuário, plano e assinatura está em `usuarios`: `premium`, `stripe_customer_id` e `stripe_subscription_id`. Não existe uma tabela de planos ou de limites; a cota gratuita é calculada por contagem em `posts`. O script `db/001_dominio.sql` usa explicitamente o nome de banco `cloud_atividade`, que deve ser alinhado ao `DB_NAME` da implantação.

3.6 Segurança

Há três papéis, atribuídos e verificados pelo auth-service: `criador` (padrão no cadastro), `revisor` e `admin`. A autorização é aplicada no backend, nas rotas, e não na interface.

Ação	admin	criador	revisor
criar, gerar, editar e excluir os próprios posts e marcas	sim	sim	não (403)
ver os próprios posts	sim	sim	sim
ver, editar ou excluir posts de outro usuário	sim	não (403)	não (403)
listar usuários e consultar o log de auditoria	sim	não (403)	não (403)
assinar o plano Creator Pro	sim	sim	sim

O isolamento por usuário é garantido nas consultas: a listagem usa `WHERE usuario_id = ?` com o identificador da sessão, e a leitura, a edição e a exclusão de um post comparam o dono com o usuário autenticado e respondem 403 quando diferem, com exceção do administrador. As consultas são parametrizadas. Cada negação gera um evento `tentativa_negada_403` no log-service. Os segredos ficam em variáveis de ambiente da stack; o repositório traz um `.env.example` apenas com os nomes. Pendências: o Compose de produção ainda contém valores padrão para segredos, que devem ser trocados, o HTTPS

com cookie seguro precisa ser configurado (`COOKIE_SECURE`) e o isolamento deve continuar sendo testado com mais de uma conta.

3.7 Observabilidade, CI/CD e documentação da API

Foram feitas as extras: health checks nos containers, métricas Prometheus com painel no Grafana, documentação OpenAPI/Swagger e um pipeline do GitHub Actions que instala dependências, executa os testes dos três serviços, valida o build e publica as imagens no GHCR; o Watchtower está previsto para atualizar as imagens. Não foi confirmada uma execução recente da implantação pública a partir desse pipeline.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no seu projeto	Quem gerencia
IaaS	a VM, a rede e o disco onde a stack roda e o servidor do MariaDB externo	Fatec / professor (infraestrutura da disciplina)
PaaS	Docker e Portainer como plataforma de execução; GitHub Actions e GHCR para build e entrega; Redis e MinIO operados como containers	eu, sobre a plataforma fornecida; não é PaaS gerenciado contratado
SaaS	o PostCraft AI, para o cliente final; e os SaaS que uso: Stripe (cobrança), Gemini (IA) e GitHub	eu (produto); Stripe, Google e GitHub (fornecedores)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	15/10/2026 14:19	entregue com atraso
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	15/10/2026 14:32	entregue
9	Fork e plano de negócio	23/10/2026	06/11/2026 20:57	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	06/11/2026 21:16	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	pendente (commit e issue de entrega); funcionamento comprovado em 12/11/2026 22:28	não entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	15/10/2026 14:19
Situação	entregue com atraso
Evidência	GitHub — commit + print do checkout concluído e do usuário premium
Link	https://github.com/luisciaramicoli/postcraft-ai/commit/35adf09e2c253a4f059faf2b5a07fbc6d1aa743e

O que foi feito. No catálogo CineCloud, o botão de upgrade passou a criar uma Checkout Session do Stripe em modo de teste, e um webhook com assinatura verificada marca o usuário como premium. O benefício comprovável era a remoção do limite de favoritos e o selo Premium no perfil. Nenhum dado de cartão passa pelo servidor.

Registro do commit `35adf09e2c25`: data 15/10/2026 14:19:35 (UTC−03:00), assunto “feat(stripe): implementa plano premium com checkout, webhooks e beneficios reais (atividade 7)”.

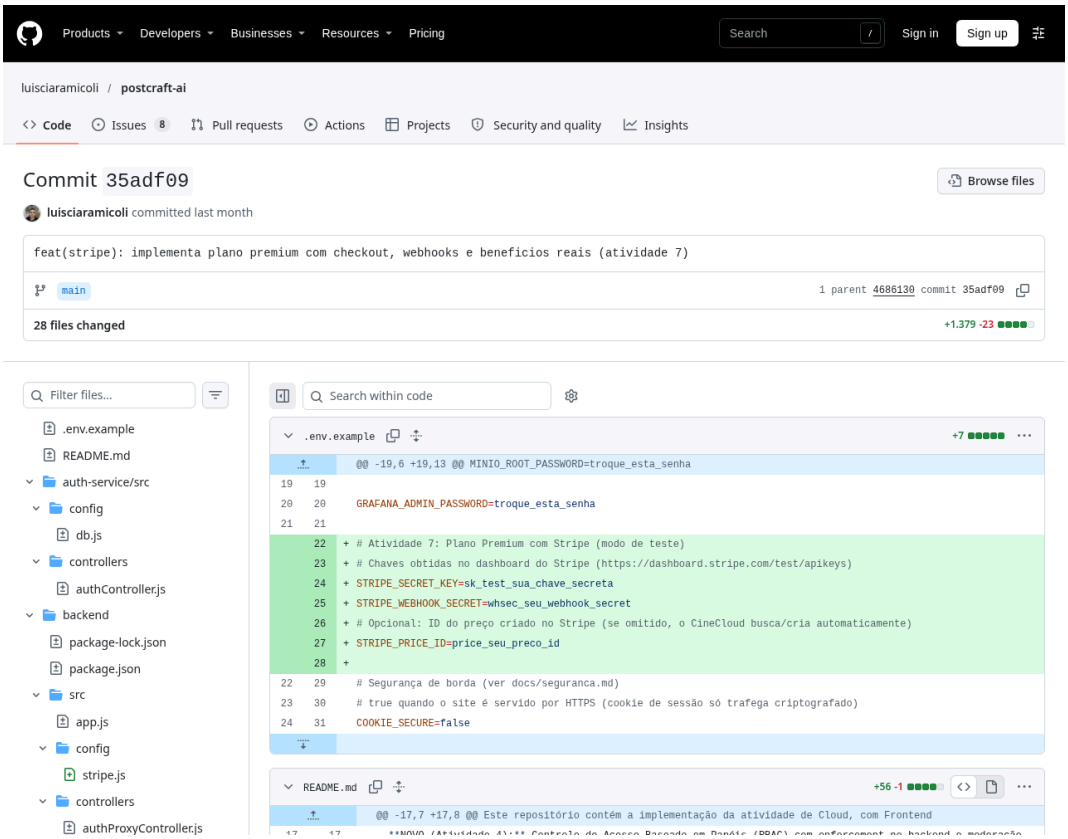


Figura 4: Atividade 7 — evidência da entrega (commit no GitHub; datas absolutas na figura de comprovação abaixo)

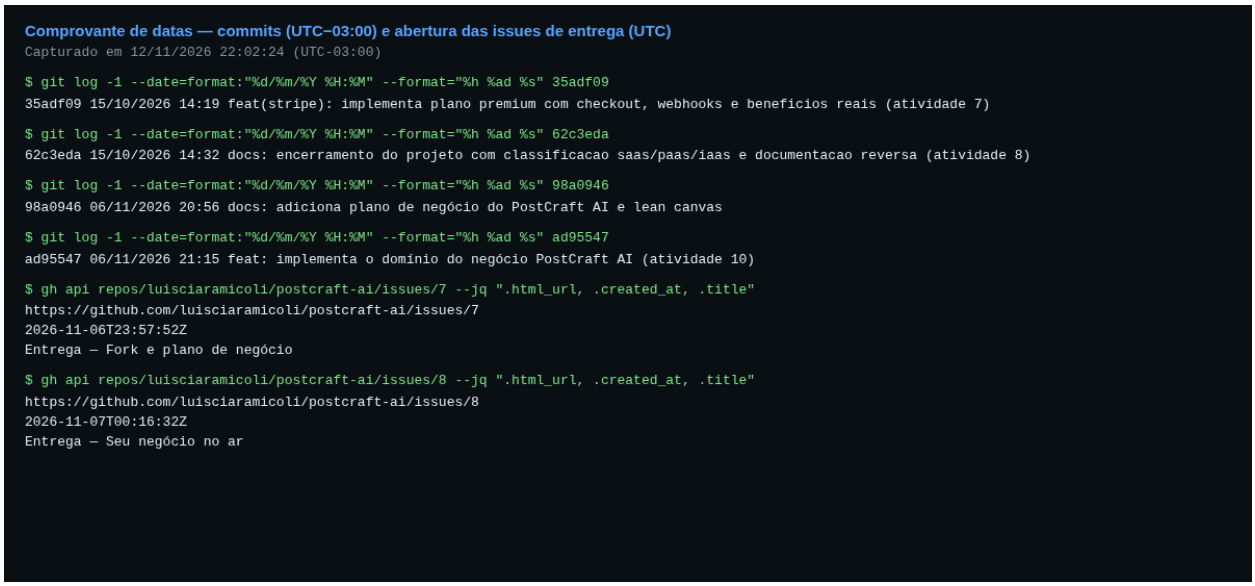


Figura 5: Comprovação de datas: `git log` dos commits das atividades 7 a 10 (horário local) e `created_at` das issues 7 e 8

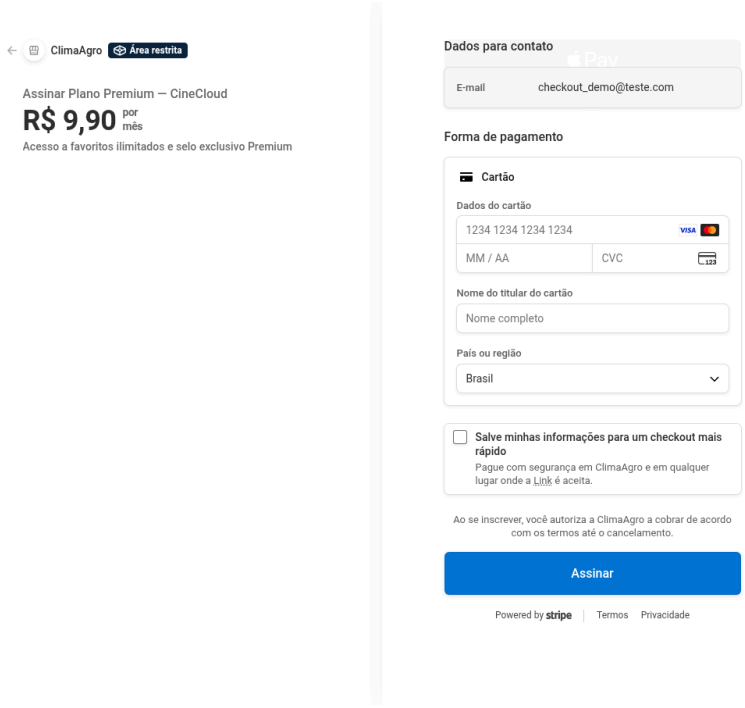


Figura 6: Atividade 7 — resultado: checkout do Stripe em modo de teste (evidência histórica de 15/10/2026)

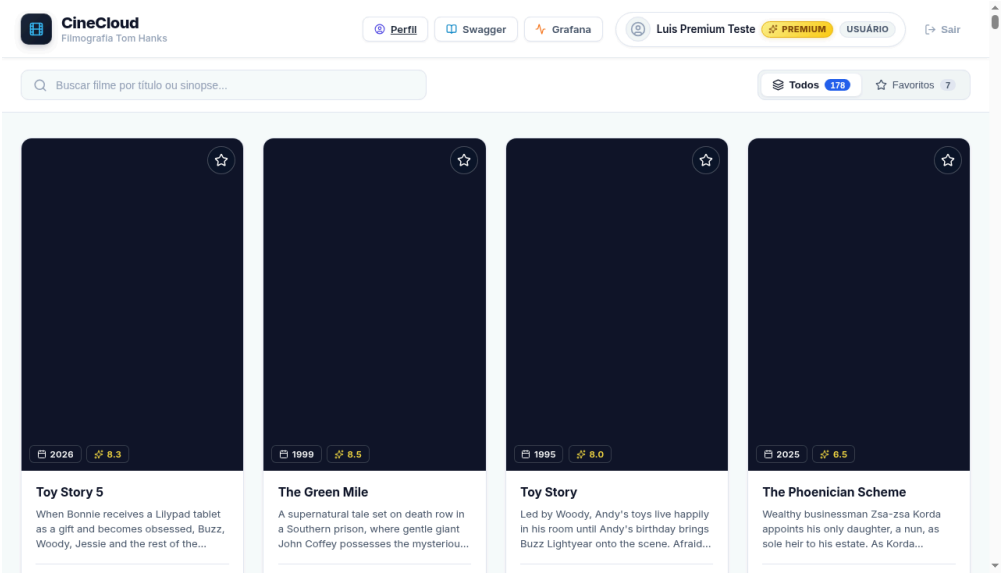


Figura 7: Atividade 7 — resultado: benefício premium no CineCloud (evidência histórica de 15/10/2026)

Dificuldades e como foram resolvidas. A entrega ocorreu depois do prazo de 09/10, e por isso conta como entregue com atraso. As imagens do resultado são do CineCloud original e comprovam o contexto da atividade, não uma assinatura do PostCraft; a assinatura do PostCraft foi testada na atividade 11.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	15/10/2026 14:32
Situação	entregue
Evidência	GitHub — commit + DOCUMENTACAO.md linkado no README
Link	https://github.com/luisciaramicoli/postcraft-ai/commit/62c3edaa8c0213da71c467e1648fe13843350387

O que foi feito. Escrevi a documentação reversa do sistema em `DOCUMENTACAO.md`, com especificação funcional, arquitetura distribuída, regras de negócio no formato “se X, então Y”, modelo de dados e a classificação da pilha em SaaS, PaaS e IaaS, e liguei o arquivo ao README. O documento descreve o CineCloud; as diferenças do domínio PostCraft estão na seção de arquitetura deste relatório.

Registro do commit `62c3edaa8c02`: data 15/10/2026 14:32:13 (UTC−03:00), assunto “docs: encerramento do projeto com classificacao saas/paas/iaas e documentacao reversa (atividade 8)”.

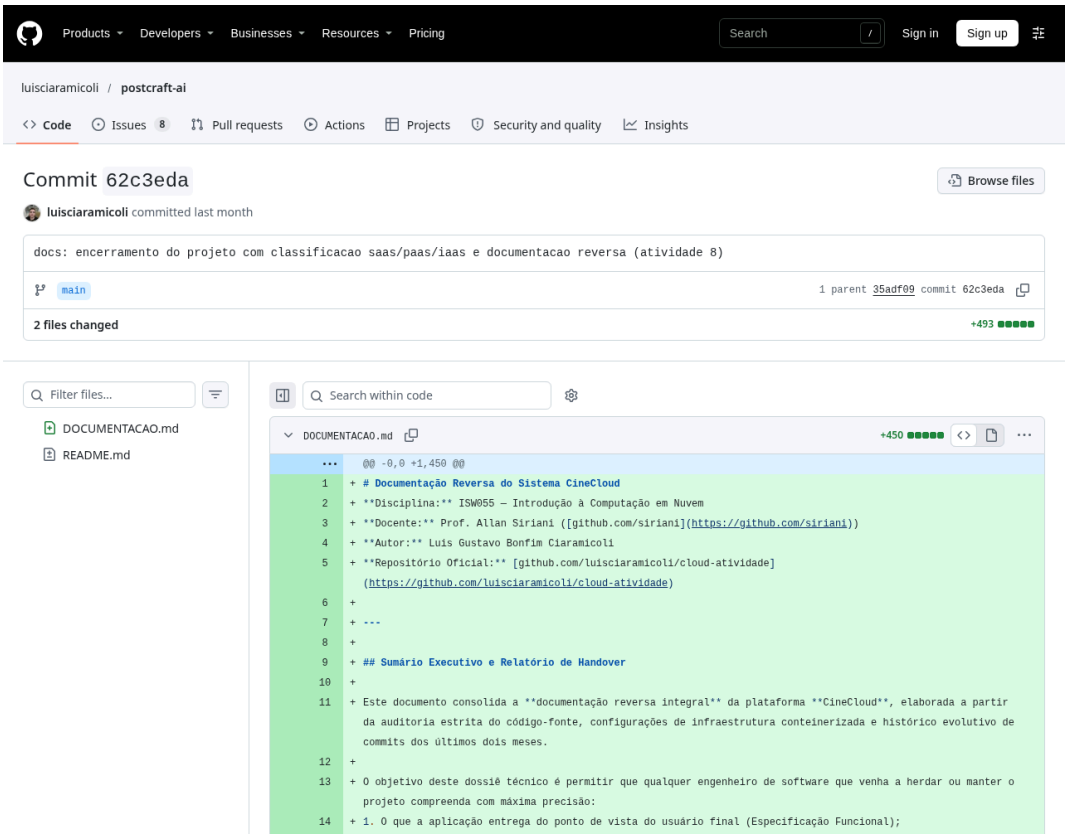


Figura 8: Atividade 8 — evidência da entrega (commit no GitHub)

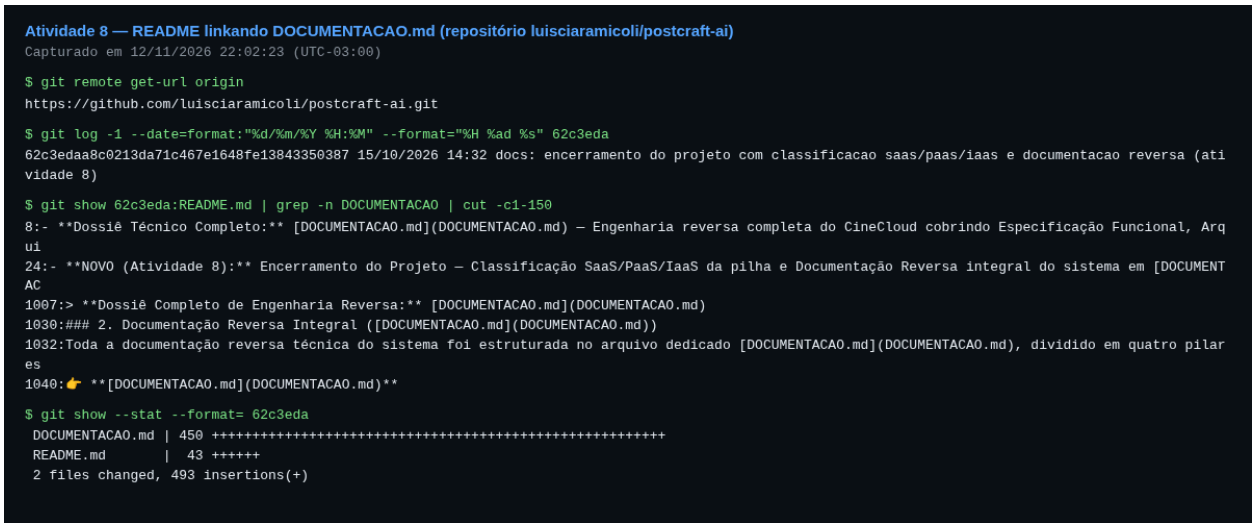


Figura 9: Atividade 8 — resultado: README do commit com os links para `DOCUMENTACAO.md`

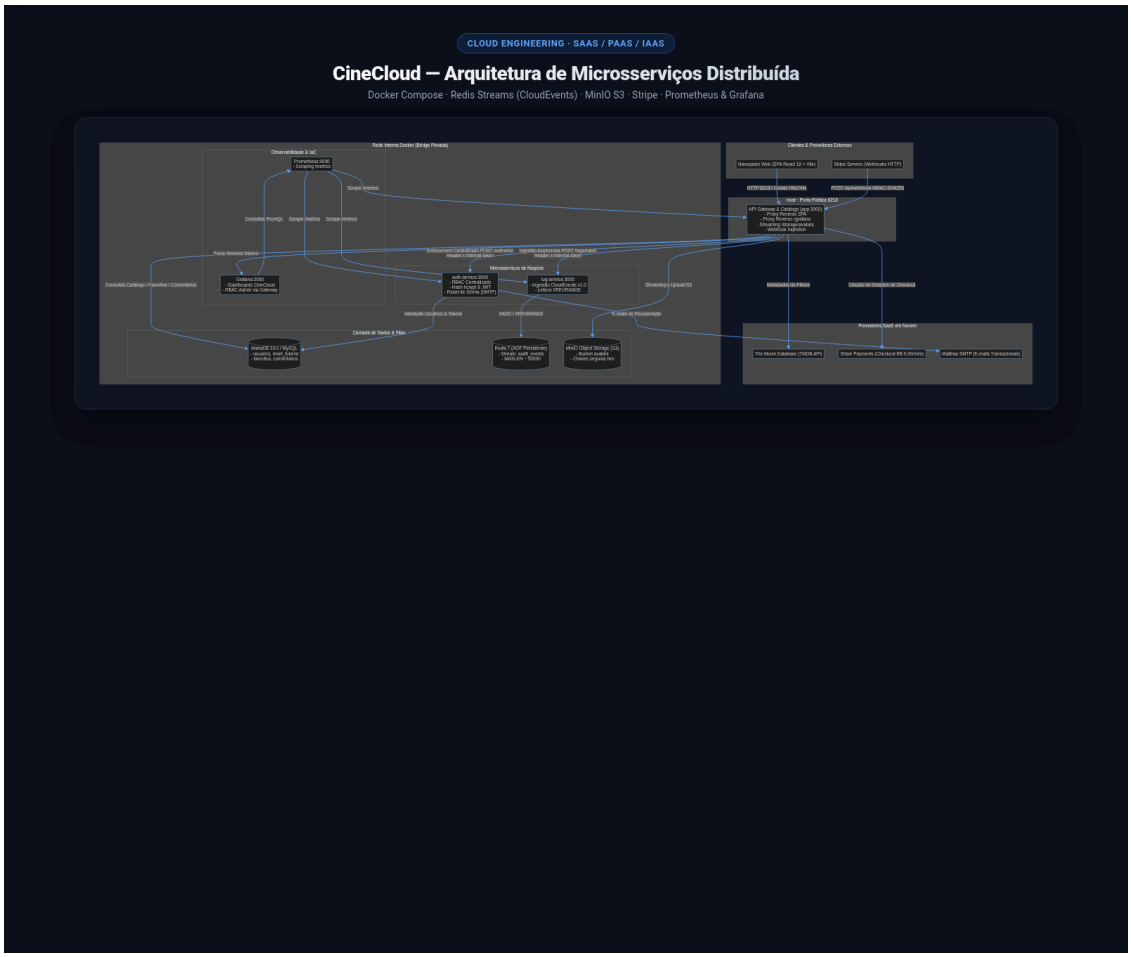


Figura 10: Atividade 8 — resultado: diagrama de arquitetura publicado na documentação

Dificuldades e como foram resolvidas. O principal cuidado foi não descrever como existente o que ainda não existia; o documento original retrata o CineCloud, e a diferença para o produto atual foi tratada na seção 3 deste relatório.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	06/11/2026 20:57
Situação	entregue com atraso
Evidência	GitHub — issue de entrega (@siriani) + repositório novo + README do produto + print do histórico de commits e das issues
Link	https://github.com/luisciaramicoli/postcraft-ai/issues/7

O que foi feito. Criei o repositório `postcraft-ai` com todo o histórico do catálogo (33 commits anteriores ao da atividade, incluindo os das atividades 7 e 8) e escrevi o plano do negócio: proposta de valor, persona, modelo de receita (gratuito com limite e Creator Pro), Lean Canvas e mapa de reaproveitamento dos serviços do CineCloud. O backlog ficou em issues. A issue de entrega, que menciona o professor, foi aberta em 06/11/2026 às 20:57 (23:57 UTC), e o commit do plano é `98a09461ee1a`, de 06/11/2026 20:56:48.

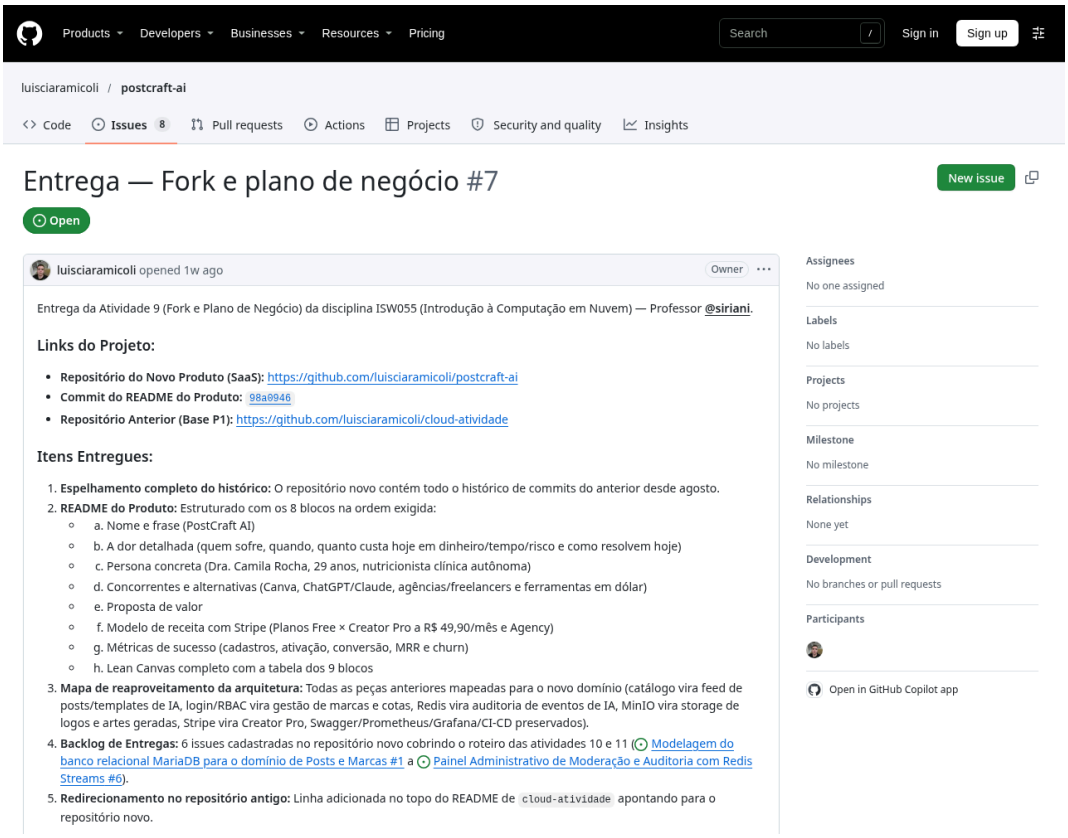


Figura 11: Atividade 9 — evidência da entrega (issue de entrega no GitHub)

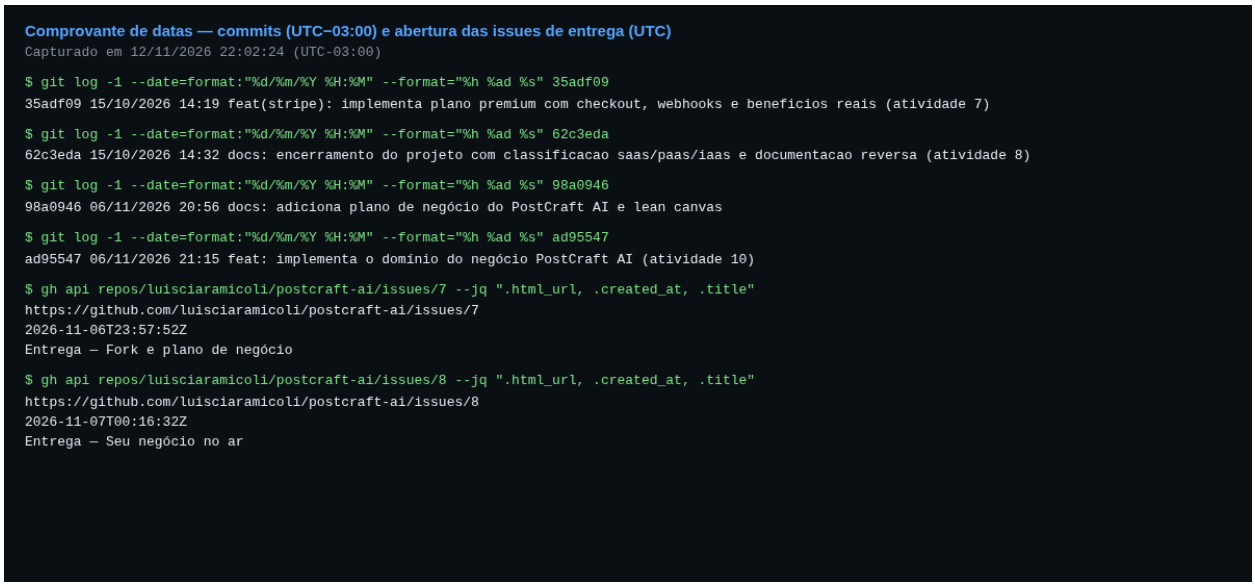


Figura 12: Atividade 9 — resultado: histórico de commits e datas das issues no repositório novo

Dificuldades e como foram resolvidas. Motivo do atraso: o repositório novo e a cópia do histórico só foram registrados na issue em 06/11, depois do prazo de 23/10. O histórico foi preservado integralmente, de modo que os commits das atividades 7 e 8 continuam localizáveis com o mesmo hash.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.

Data planejada	06/11/2026
Data realizada	06/11/2026 21:16
Situação	entregue
Evidência	GitHub — issue de entrega (@siriani) + commit + URL no ar + prints da landing, do 403 e dos eventos no log
Link	https://github.com/luisciaramicoli/postcraft-ai/issues/8

O que foi feito. Troquei o domínio de filmes pelo de posts e marcas: tabelas `marcas`, `posts` e `templates`, API do negócio, papéis `criador`, `revisor` e `admin`, isolamento por usuário nas consultas, eventos de negação no log-service, upload de logo e de foto no MinIO e a landing page. O commit é `ad95547d8667`, de 06/11/2026 21:15:56 (UTC−03:00); a issue de entrega foi aberta às 21:16 (00:16 UTC de 07/11).

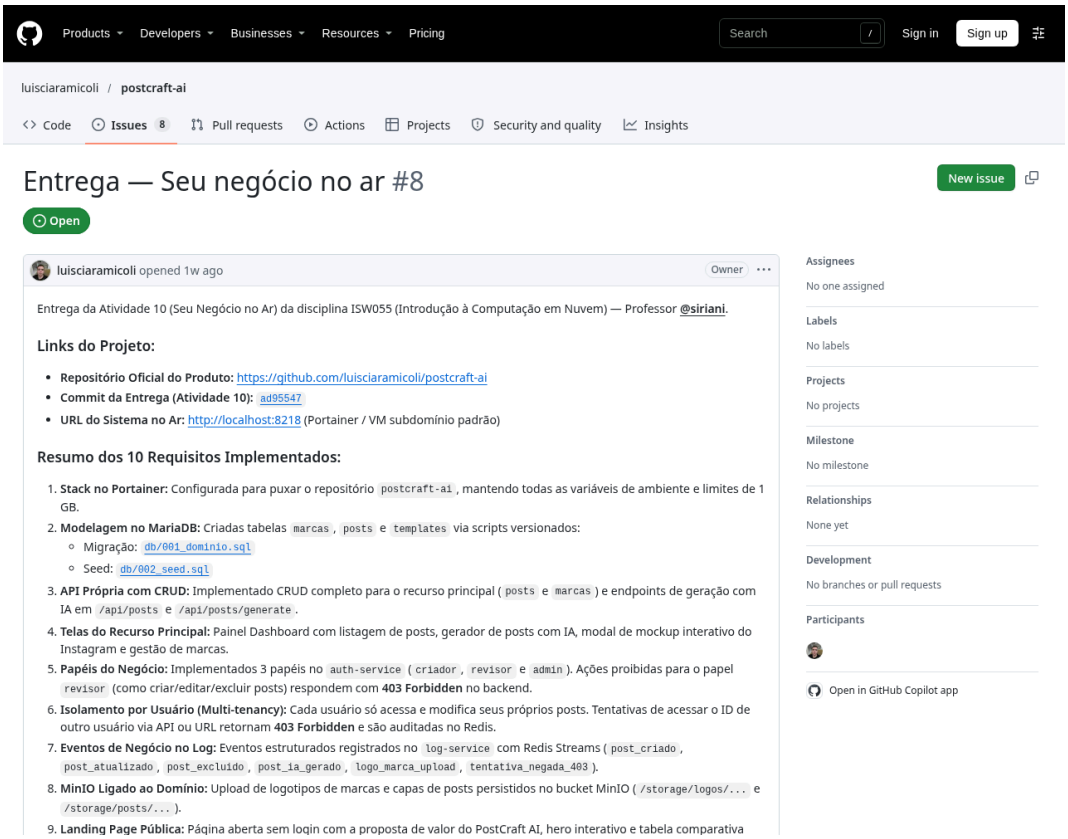


Figura 13: Atividade 10 — evidência da entrega (issue de entrega no GitHub)



Figura 14: Atividade 10 — resultado: página inicial pública, capturada em 12/11/2026 às 21:56

Pretty-print

{"status":"healthy","service":"catalog-service","checks":{"database":"connected"},"timestamp":"2026-11-13T00:56:56.258Z"}

Figura 15: Atividade 10 — resultado: `/api/health` da URL pública (timestamp 2026-11-13T00:56:56Z, igual a 12/11/2026 21:56:56 em UTC−03:00)



Figura 16: Atividade 10 — resultado: 403 Forbidden no acesso e na exclusão de post de outro usuário e na rota de administração, com a conta B



Figura 17: Atividade 10 — resultado: eventos `tentativa_negada_403` e `cross_tenant_denied` no log-service (Redis Stream `audit_events`)

Dificuldades e como foram resolvidas. A issue de entrega informava apenas `localhost` como endereço; o site público só foi verificado depois, em 12/11/2026. Os prints do 403 e dos eventos de log foram feitos nessa data na stack local (porta 8218), com duas contas de teste do papel criador, e não no domínio público. A entrega do código foi no prazo.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	pendente (commit e issue de entrega); funcionamento comprovado em 12/11/2026 22:28
Situação	não entregue
Evidência	GitHub — issue de entrega (@siriani) + commit + prints da assinatura com o cartão 4242, do Stripe em modo de teste, do webhook e do limite bloqueando
Link	—

O que foi feito. Implementei o limite do plano gratuito no servidor: três posts gerados por mês, verificados antes de chamar o Gemini, com 403 e o código `limite_gratuito_atingido`; usuários premium não têm limite. O limite é coberto por teste automatizado (`tests/quota.test.js`, 3 testes). Em 12/11/2026, entre 22:23 e 22:28, executei o ciclo completo de assinatura numa conta de teste, com o Stripe em modo de teste e o cartão 4242: (1) no plano gratuito, a quarta geração foi bloqueada; (2) o checkout hospedado do Stripe aceitou o cartão 4242; (3) o webhook ativou o plano em cerca de 1 segundo e a quarta geração foi aceita; (4) o cancelamento da assinatura, às 22:28:06, devolveu a conta ao plano gratuito em cerca de 3 segundos e o bloqueio voltou. O teste rodou numa instância local do app, com o código que contém a cota, ainda sem commit; o webhook foi entregue pelo Stripe ao domínio público, e o app público gravou no banco compartilhado, o que é uma inferência pelo comportamento observado.

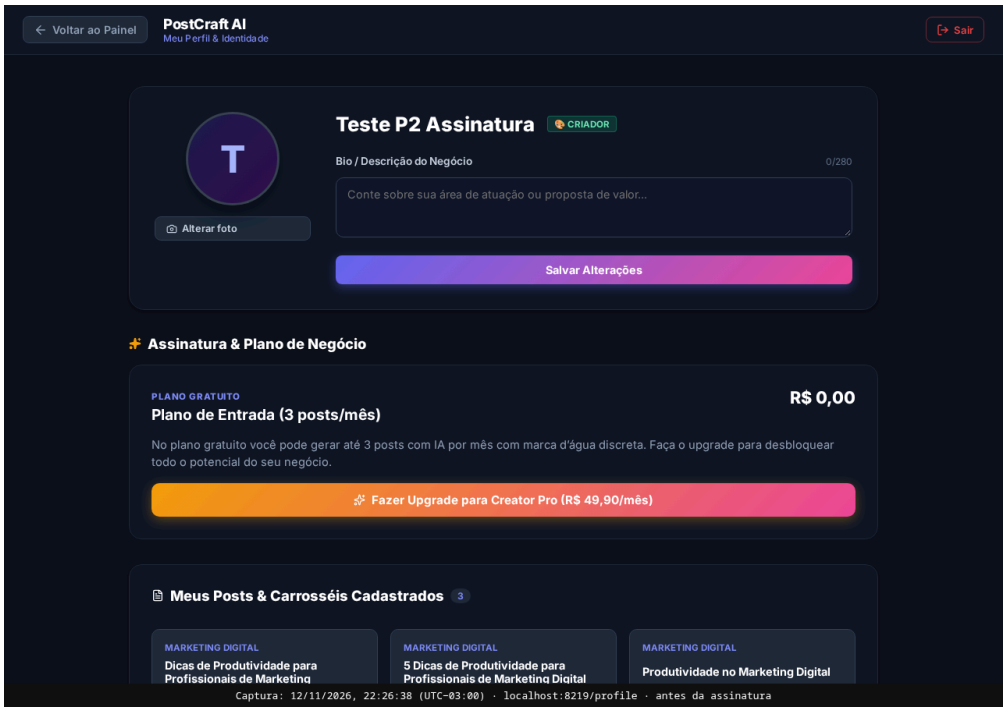


Figura 18: Atividade 11 — passo 1: perfil no plano gratuito, antes da assinatura (12/11/2026 22:26)

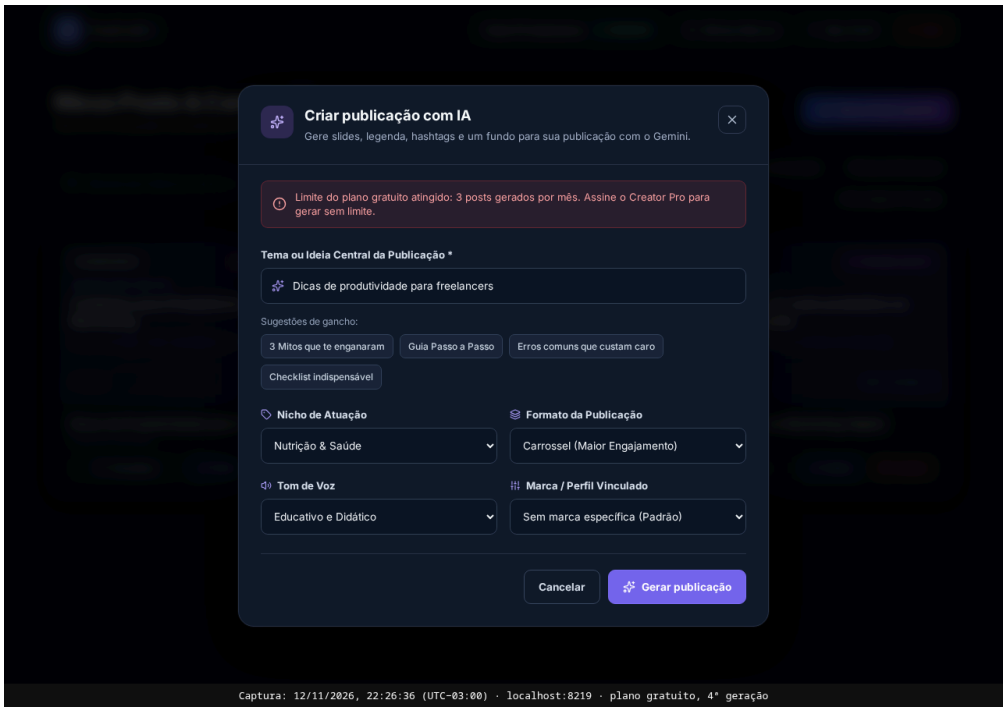


Figura 19: Atividade 11 — passo 1: quarta geração do mês bloqueada na interface (12/11/2026 22:26)

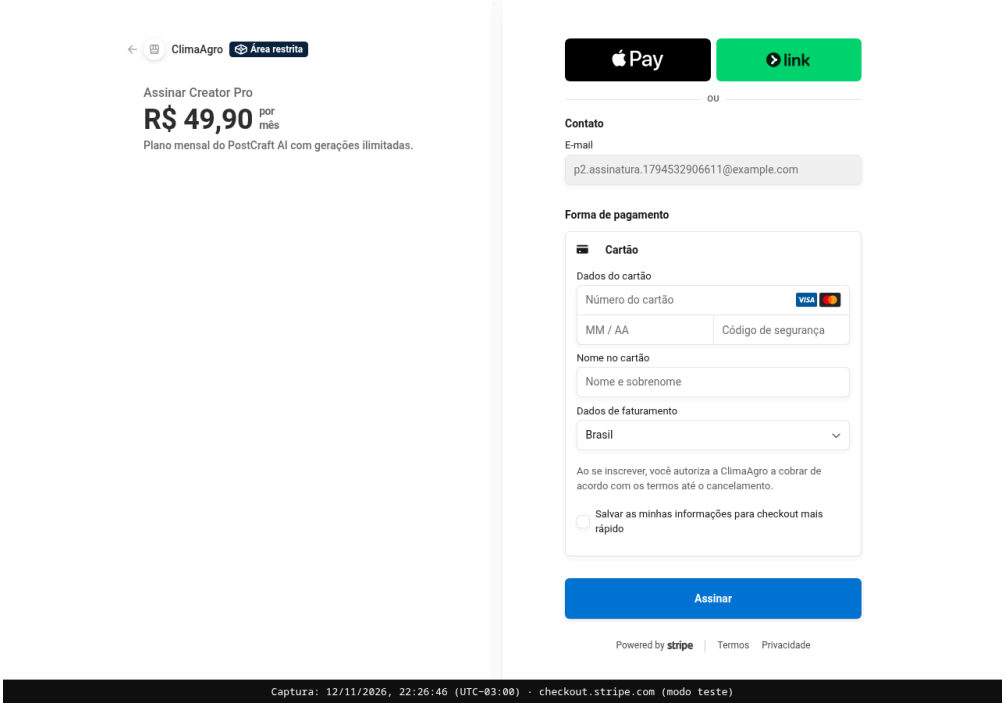


Figura 20: Atividade 11 — passo 2: checkout hospedado do Stripe, modo de teste, plano Creator Pro (12/11/2026 22:25)

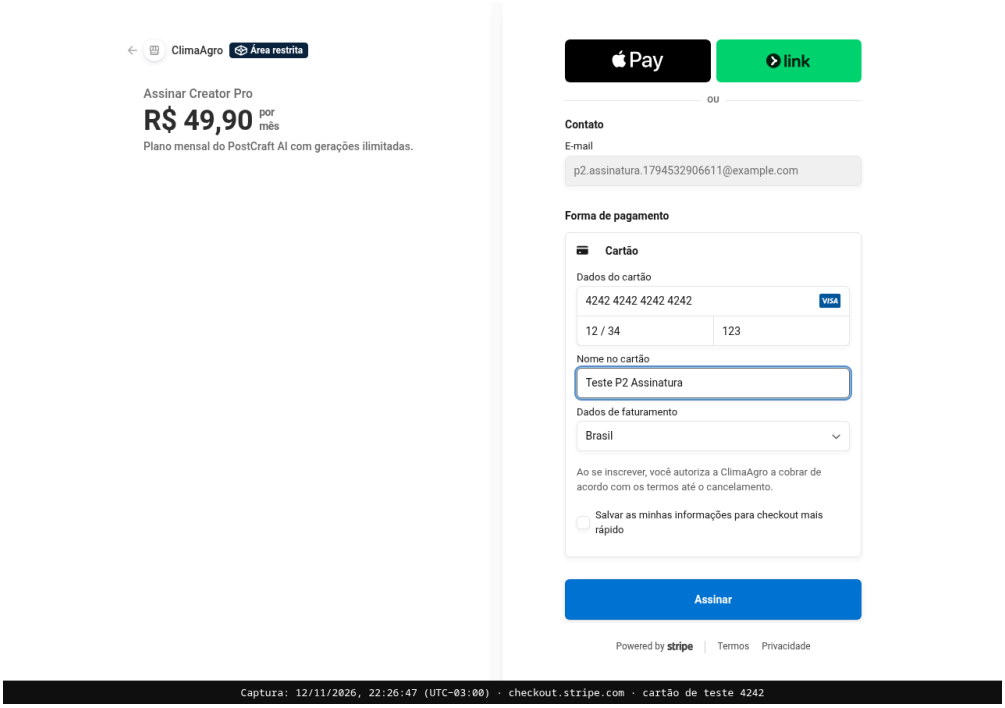


Figura 21: Atividade 11 — passo 2: cartão de teste 4242 preenchido antes de confirmar (12/11/2026 22:26)

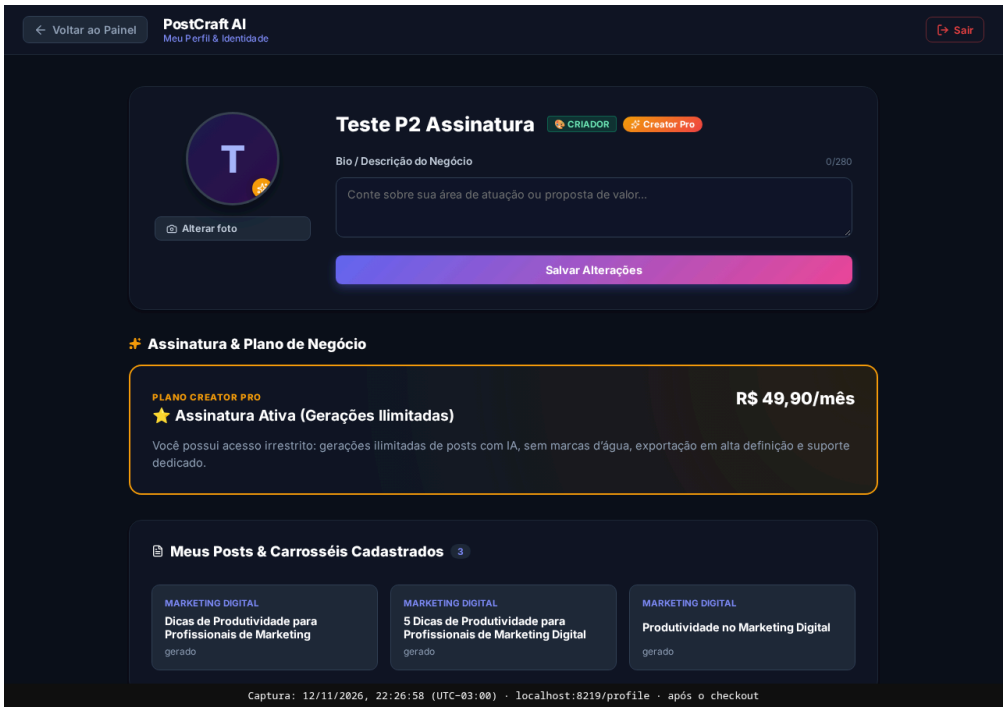


Figura 22: Atividade 11 — passo 3: plano Creator Pro ativo após o webhook (12/11/2026 22:26)

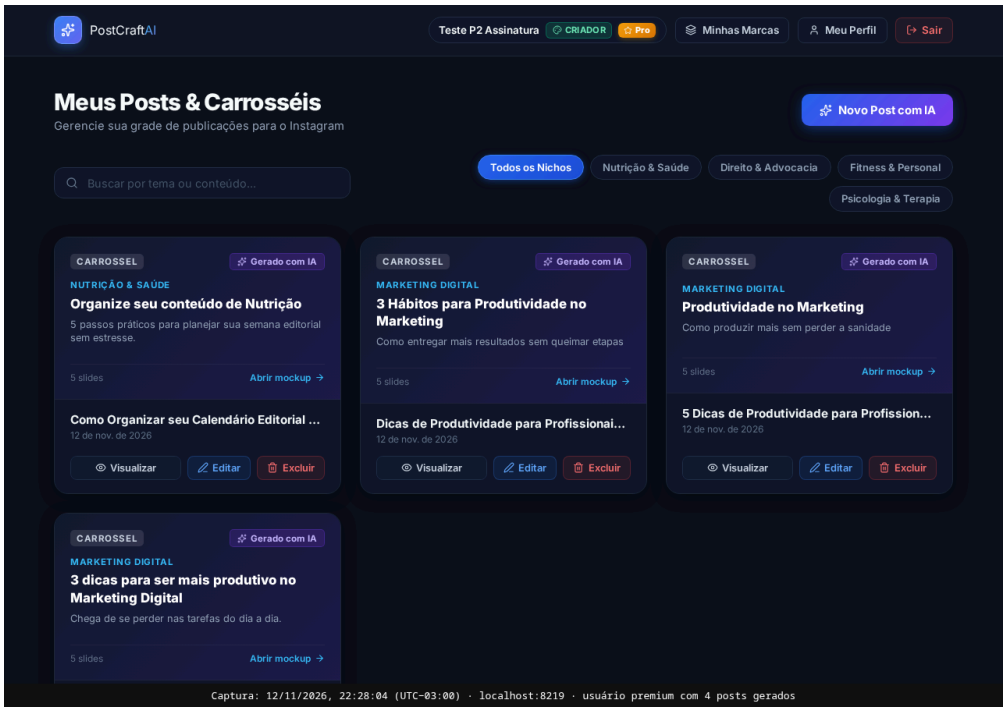


Figura 23: Atividade 11 — passo 3: conta premium com o quarto post gerado, acima do limite gratuito (12/11/2026 22:28)

Eventos Stripe do cliente de teste (modo teste, consulta via API GET /v1/events)

Cliente cus_VcO2… · usuário de teste criado para esta validação · horários America/Sao_Paulo (UTC−03:00)

Criado em	Tipo do evento	ID do evento	pending_webhooks
12/11/2026, 22:26:49	customer.subscription.created	evt_1Ub9GRD9qoJtG1qnDzw5qRpy	0
12/11/2026, 22:26:51	checkout.session.completed	evt_1Ub9GRD9qoJtG1qnhFTc6xHy	0
12/11/2026, 22:28:06	customer.subscription.deleted	evt_1Ub9HeD9qoJtG1qnskzJbt9g	0

pending_webhooks = 0: o Stripe não tem entregas pendentes para o endpoint cadastrado (https://luis-ciaramicoli-isw055.lapps.studio/api/webhook).

Figura 24: Atividade 11 — webhook: eventos do cliente de teste no Stripe, com `pending_webhooks = 0` (consulta à API)

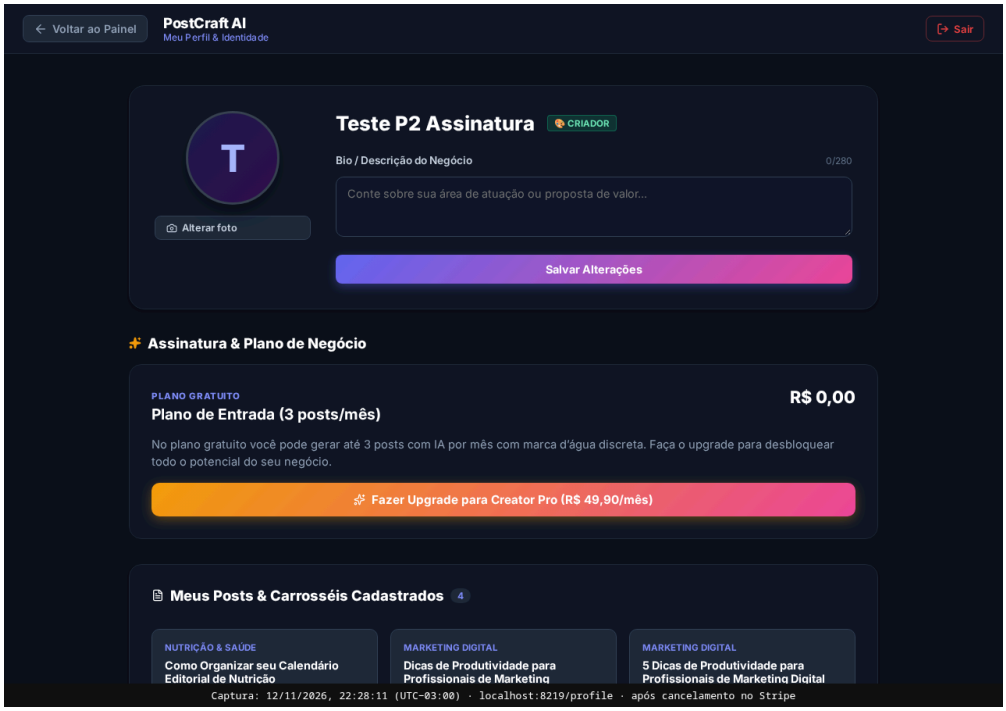


Figura 25: Atividade 11 — passo 4: perfil de volta ao plano gratuito após o cancelamento (12/11/2026 22:28)

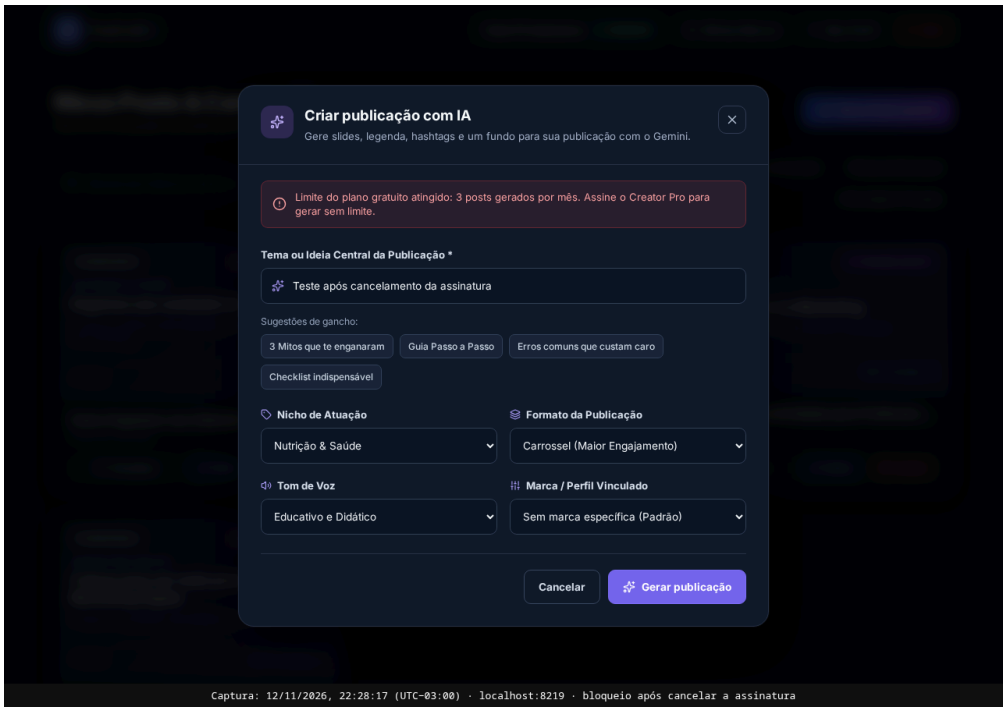


Figura 26: Atividade 11 — passo 4: geração bloqueada novamente após o cancelamento (12/11/2026 22:28)

Atividade 11 — limite do plano gratuito (teste automatizado, sem Stripe real)
Capturado em 12/11/2026 22:03:02 (UTC-03:00)

```
$ trecho de backend/src/controllers/postController.js
const FREE_MONTHLY_LIMIT = 3;

// Plano gratuito: no máximo FREE_MONTHLY_LIMIT posts gerados por mês (premium é ilimitado).
// A verificação lê o campo premium no banco (atualizado pelo webhook do Stripe), não o JWT.
async function freeQuotaExceeded(userId) {
  const [users] = await db.execute('SELECT premium FROM usuarios WHERE id = ?', [userId]);
  if (users[0] && users[0].premium) return false;
  const [rows] = await db.execute(
    "SELECT COUNT(*) AS total FROM posts WHERE usuario_id = ? AND status IN ('gerado', 'publicado') AND criado_em >= DATE_FORMAT(NOW(), '%Y-%m-0
1')",
    [userId]
  );
  return Number(rows[0].total) >= FREE_MONTHLY_LIMIT;
}

$ node --test tests/quota.test.js
✔ plano gratuito com 3 gerações no mês é bloqueado com 403 antes de chamar o Gemini (1.418115ms)
✔ plano gratuito abaixo do limite segue para a geração (1.073454ms)
✔ usuário premium não é limitado (0.309357ms)
ℹ tests 3
ℹ pass 3
ℹ fail 0
```

Figura 27: Atividade 11 — teste automatizado do limite: `freeQuotaExceeded` e resultado de `tests/quota.test.js` (banco e Gemini simulados)

Dificuldades e como foram resolvidas. A atividade consta como não entregue porque o commit com o limite de cota e a issue de entrega ainda não foram registrados. Também não estão prontos o portal do cliente (o cancelamento foi feito pela API do Stripe), o tratamento de eventos repetidos, o painel admin de assinaturas e a publicação do limite no site público, que ainda roda a imagem anterior, sem a cota. Num teste anterior, o checkout do Stripe retornou erro de credencial expirada; o erro não se repetiu em 12/11 e a causa não foi investigada. O Stripe exibe o nome de negócio ClimaAgro porque a conta é compartilhada com outro projeto.

6 Considerações finais

Neste bimestre aprendi a transformar um catálogo em um produto com cobrança, armazenamento, logs e domínio próprio, e, principalmente, a separar o que o código faz do que a interface apenas promete. Um botão de plano ou de geração não prova que a cobrança recorrente, a autorização e a cota funcionam; só o ciclo completo, da assinatura ao cancelamento, com um webhook de verdade, prova. O mais difícil foi manter versões diferentes alinhadas: o produto, a base de código, a imagem publicada e o que a stack pública executa nem sempre são a mesma coisa.

Faria diferente em três pontos: registraria a entrega de cada atividade no dia, com issue e commit, em vez de acumular; trataria a idempotência do webhook e o portal do cliente desde o primeiro desenho da assinatura; e guardaria os prints a cada marco, em vez de reconstruí-los depois.

Para ser um produto de verdade, o PostCraft AI ainda precisa de: commit do limite de cota e da issue de entrega da atividade 11; publicação dessa versão no site público; portal do cliente e deduplicação de eventos de webhook; tratamento de falha de pagamento; troca dos segredos padrão do Compose de produção e HTTPS com cookie seguro; limites de memória nos containers; e testes de isolamento com mais de uma conta e papel.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. A redação e a montagem tiveram assistência de IA, como declarado na metodologia.

Luis Gustavo Bonfim Ciaramicoli

Pompeia, 12/11/2026